

Restaurant Portal — Complete Project Walkthrough

1. Project Objective

The Restaurant Portal is a complete restaurant operations application for a kitchen team of 12 people. It allows staff to view weekly working timings, identify weekly days off, see the daily meal and dessert, and submit meal requests.

The application uses:

- React for the frontend
- Flask for the backend API
- SQLite for automatic database persistence
- Vite for development and production bundling
- Git and GitHub for version control and deployment

No Docker is required.

2. Team Structure

The initial database contains 12 employees:

Role	Team members
Manager	2
Chef de cuisine	2
Chef de second	2
Commis de cuisine	6
Total	12

The database is seeded automatically when Flask starts for the first time.

3. Application Architecture

The project has two main parts:

1. Frontend

- React application created with Vite
- Displays the dashboard, weekly schedule, daily menu, and meal requests
- Sends requests to the Flask API using the browser's Fetch API

2. Backend

- Flask application provides REST endpoints
- Uses SQLite to store application data
- Serves the production frontend when it has been built

The React frontend is connected to the Flask API through `/api` URLs. The development server uses a proxy to send these requests to Flask on port 5000.

4. Database Design

The SQLite database is stored in `restaurant.db`.

The main tables are:

Employees

Stores the employee name, role, initials, phone number, and email.

Weekly Shifts

Stores one row per employee and day. Each row contains:

- Employee ID
- Week start date
- Weekday
- Shift type
- Start time
- End time
- Off-day status

The initial application creates 84 rows: 12 employees multiplied by 7 days.

Daily Menu

Stores the meal and dessert for each date. The date is unique, so each day has one menu record.

Meal Requests

Stores employee meal requests, including the requested date, meal, optional note, status, and creation time.

5. Backend API

The Flask application provides the following endpoints:

Method	Endpoint	Purpose
GET	/api/health	Checks whether the application is running
GET	/api/staff	Retrieves all team members
GET	/api/schedule? week_start=YYYY-MM-DD	Retrieves weekly shifts for a selected week
GET	/api/menu	Retrieves the latest seven daily menus
POST	/api/meal-requests	Creates a meal request
GET	/api/meal-requests	Retrieves all meal requests
GET	/	Serves the production frontend

The meal-request endpoint validates that the employee ID, request date, and meal are present. It stores the request with a default status of **Pending**.

6. Frontend Features

The React dashboard contains four sections.

Overview

The overview displays:

- Today's daily meal and dessert
- Total number of team members
- Number of shifts this week
- Number of days off
- Kitchen role distribution
- Recent meal requests
- Recent daily menu information

Weekly Schedule

The weekly schedule displays every employee across Monday through Sunday. Each employee has a shift time or an off-day indicator.

The user can move between weeks using the previous, current-week, and next controls.

Daily Menu

The daily menu page displays the latest seven menus. Each record shows the meal, dessert, date, and

update time.

Meal Requests

Employees can submit a new meal request through a modal form. The request can include:

- Employee
- Requested date
- Meal preference
- Optional allergy or special-request note

The request is saved in SQLite and appears in the requests section.

7. Database Communication

Database communication is automatic because the frontend calls Flask through REST endpoints.

A typical meal-request flow is:

1. The user selects an employee and enters a meal request.
2. React sends a POST request to `/api/meal-requests`.
3. Flask validates the request.
4. Flask inserts the request into SQLite.
5. Flask returns the saved request as JSON.
6. React updates the request list.

The schedule and menu pages also reload the database through API requests. Therefore, no manual database refresh is required inside the browser.

8. Local Setup

Backend

Create and activate a Python virtual environment:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python backend/app.py
```

The backend runs on:

```
http://127.0.0.1:5000
```

SQLite is created automatically as `restaurant.db`.

Frontend

Run the React development server:

```
cd frontend  
npm install  
npm run dev
```

Open:

```
http://127.0.0.1:5173
```

Vite proxies `/api` requests to Flask on port 5000.

9. Production Build

Build the frontend:

```
cd frontend  
npm install  
npm run build
```

The production files are written to `frontend/dist`.

Flask serves the generated frontend and its JavaScript and CSS files. The application can then be started with:

```
python backend/app.py
```

No Docker is required.

10. Deployment on Linux

The application is designed to run as a normal Flask application.

The deployment steps are:

1. Clone the repository.
2. Create a Python virtual environment.
3. Install the dependencies from `requirements.txt`.
4. Build the React frontend with `npm run build`.
5. Start Flask with `python backend/app.py`.
6. Use a reverse proxy such as Nginx or Caddy if the application must be exposed publicly.

For production, the development server should be replaced with a production WSGI server such as Gunicorn. The project currently uses Flask's development server for simple local and server deployment.

11. Git and GitHub

The project was initialized as its own Git repository.

The repository was committed with:

```
51bfe8f Initial restaurant portal
```

The remote is:

```
https://github.com/harishnaakidi93-hash/beginner-git.git
```

The local branch is `master`, and it is configured to track `origin/master`.

The repository can be updated later with:

```
git status
git add .
git commit -m "Update restaurant portal"
git push
```

12. Important Notes

- The database is ignored by Git because it contains runtime data.
- The frontend build output is also ignored by Git.
- The initial database contains sample team and schedule data.
- The application is not yet using authentication or user accounts.
- Meal requests are currently available to all staff because the portal does not yet implement login roles.
- The current production deployment uses Flask's development server; a production WSGI server can be added later.
- The SQLite database should be backed up periodically if it will be used in a shared production environment.

13. Project Files

- `backend/app.py` — Flask API and database setup
- `frontend/src/App.jsx` — React application and UI
- `frontend/src/styles.css` — responsive application styling
- `frontend/package.json` — React and Vite configuration
- `frontend/vite.config.js` — frontend development proxy configuration
- `requirements.txt` — Python dependencies

- `README.md` — local setup instructions
- `DEPLOYMENT.md` — deployment instructions
- `PROJECT_WALKTHROUGH.md` — this document